

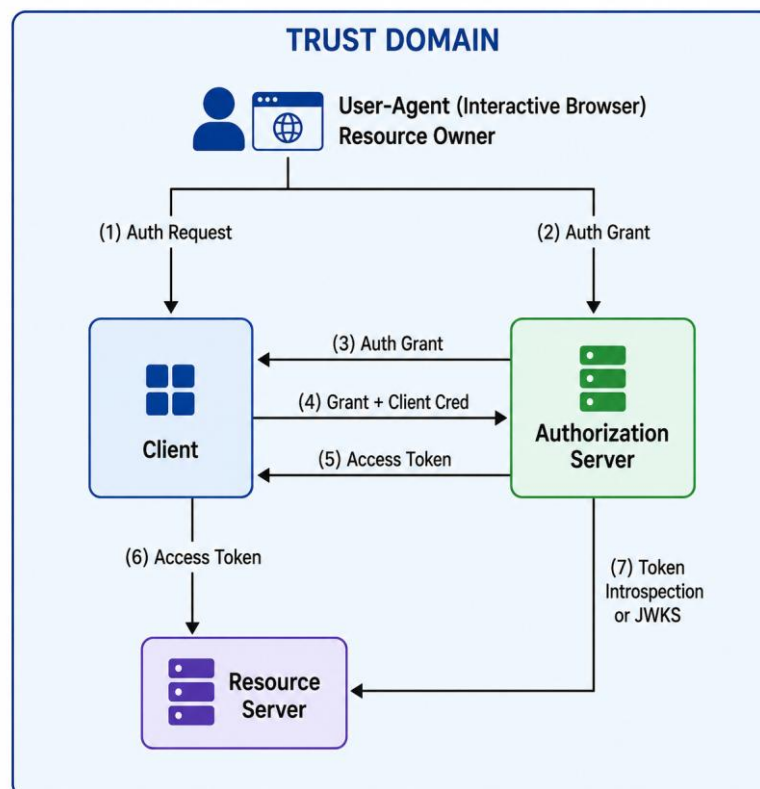
Phase 1: Theoretical Foundations of Modern Authentication and Authorization Protocols

Architectural Breakdown of the Four Core OAuth 2.0 Actors (RFC 6749)

The OAuth 2.0 authorization framework (RFC 6749) structurally separates the authentication of a user's identity from the authorization to access protected web resources.¹ Historically, distributed systems relied on credential-sharing patterns, requiring client applications to collect and store user passwords.⁴ This design violated basic security tenets by exposing raw credentials to third-party applications, expanding the password-compromise blast radius, and preventing fine-grained access control.⁴ By shifting to an assertion-based token delegation model, OAuth 2.0 establishes distinct trust domains where a single Authorization Server manages access policies across a group of Resource Servers.³

The Trust Domain Architecture

A trust domain constitutes the boundary managed by a single Authorization Server.⁶ Within this domain, a client can move freely with a single access token, provided the Resource Servers are configured to validate tokens emitted by that specific authority.⁶ Decoupling authentication from authorization relies on coordinated interactions among four distinct architectural actors, each operating within strict cryptographic and physical trust boundaries.¹



Resource Owner

The Resource Owner is the end-user or system entity that holds the authoritative rights over the protected data and is capable of granting access to a protected resource.¹ The boundary of the Resource Owner is located at the interactive user-agent (typically a secure web browser or native system browser).¹⁰ This actor must never expose its authentication credentials (such as usernames, passwords, or biometrics) to the Client.⁴ Instead, the Resource Owner authenticates directly with the Authorization Server, reviewing and approving the requested access scopes in a dedicated execution space isolated from the Client application.²

Client

The Client is the application requesting access to protected resources on behalf of the Resource Owner.¹ The technical responsibility of the Client is to coordinate the redirection flows, handle authorization grants, securely store issued tokens, and present access tokens to Resource Servers.³ Clients are categorized into two structural trust profiles based on their execution environments and ability to protect cryptographic secrets¹¹:

- **Confidential Clients:** Applications executing in isolated environments where client credentials (such as client secrets or private asymmetric keys) are protected from unauthorized extraction.¹¹ Examples include backend microservices, server-rendered applications, and database-connected APIs.⁷ These clients can perform secure back-channel calls to the token endpoint and authenticate using robust methods, including HTTP Basic Auth (client_secret_basic), request parameters (client_secret_post), or asymmetric private key signatures (private_key_jwt).¹¹
- **Public Clients:** Applications running in user-controlled or distributed environments where executable code, memory spaces, and local storage can be inspected or decompiled.⁷ Examples include Single Page Applications (SPAs) running in web browsers, native mobile apps, and IoT devices.⁷ Public clients cannot maintain a client secret, making client authentication at the token endpoint impossible.⁷ Consequently, modern standards completely deprecate the Implicit Grant for these clients due to the risk of token interception via front-channel redirection.⁴ Public clients must implement the Authorization Code Flow with Proof Key for Code Exchange (PKCE) to prevent code-injection and interception attacks.² PKCE enforces a dynamic cryptographic binding between the authorization request and the token exchange request by utilizing a high-entropy code verifier and its SHA-256 challenge hash.² Mobile public clients must also execute authorization requests via external user-agents, such as system browser tabs, ensuring that session cookies and credentials remain isolated from the host application.¹⁰

Authorization Server

The Authorization Server is the centralized engine responsible for verifying the identity of the Resource Owner, managing client registrations, presenting consent interfaces, and issuing cryptographic tokens.² In enterprise architectures, this role is performed by identity providers such as Keycloak or Auth0.⁶ The Authorization Server exposes standardized endpoints, including the authorization endpoint, token endpoint, token revocation endpoint, and token introspection endpoint.² It publishes metadata via the standard discovery path /.well-

known/oauth-authorization-server or /.well-known/openid-configuration (RFC 8414), which lists supported algorithms, endpoints, and client authentication mechanisms.² This actor translates authentication events and user approvals into signed, cryptographically protected assertions.³

Resource Server

The Resource Server is the API or backend microservice hosting the protected data.¹ It receives incoming requests, extracts bearer tokens from the HTTP Authorization: Bearer <token> header¹¹, and validates those tokens before exposing endpoints.³ This validation can be achieved through two distinct architectural strategies:

- **Independent Validation:** The Resource Server parses the incoming token as a JSON Web Token (JWT).¹⁸ It fetches the public key set (JSON Web Key Set, or JWKS) from the Authorization Server's metadata URI, caches the keys locally, and validates the cryptographic signature independently.²⁰ This stateless model eliminates the need for a network round-trip on every API call, making it suitable for high-throughput microservice networks.²²
- **Dependent Validation (Token Introspection, RFC 7662):** The Resource Server performs an active, stateful validation check.⁸ It executes an authenticated HTTP POST request to the Authorization Server's introspection endpoint, sending the presented token as application/x-www-form-urlencoded data.⁸ The Authorization Server returns a JSON document containing an active boolean along with the token's metadata (issuer, scopes, expiration, and user context).⁸ This stateful model is necessary for unstructured, opaque tokens²⁴ or when the system requires real-time, high-assurance token revocation checks.²⁴ To mitigate the performance penalties of repeated network hops, systems often implement a hybrid approach, caching the introspection responses in high-speed, short-lived storage, such as Redis, with a 30-to-60-second Time-To-Live (TTL).²⁶

Token Typology and Architectural Anti-Patterns: Access Token vs. ID Token

Confusion between authentication (verifying *who* an entity is) and authorization (verifying *what* an entity is allowed to do) represents a significant security risk in distributed applications.² While OAuth 2.0 is strictly an authorization framework designed to issue access permissions for APIs², OpenID Connect (OIDC) Core 1.0 extends this framework to introduce a standardized user identity layer.² Under OIDC, successful user authentication yields two distinct token types with non-overlapping functional scopes: the Access Token and the ID Token.³

The Identity Assertion Fallacy

A common security anti-pattern involves treating Access Tokens as identity assertions in client frontends, or presenting ID Tokens to backend APIs for authorization decisions.³ This structural error—referred to as the "Identity Assertion Fallacy"—leads to several key system vulnerabilities:

- **Audience Mismatch and Token Replay:** The audience claim (aud) of an ID Token is explicitly bound to the Client's client_id.⁷ If a backend API accepts an ID Token for

authorization, it must validate this aud claim.⁶ However, because the API's identifier differs from the client_id, the API must either bypass audience validation or accept a client identifier.⁷ Bypassing audience validation introduces token replay vulnerabilities.⁴ An attacker could intercept an ID Token intended for a low-privilege client and replay it against a high-privilege API, leading to broken access control and privilege escalation.⁵

- **Lack of Authorization Context:** ID Tokens are designed to convey identity attributes (claims such as name, email, and authentication timestamps).⁷ They lack the scopes, target audiences, and resource-level permissions required to make fine-grained authorization decisions at the resource layer.⁷
- **Information Disclosure (PII Leakage):** ID Tokens frequently contain sensitive user-profile attributes (e.g., email, street address, or phone number) to personalize the client application.⁷ Forwarding the ID Token to downstream microservices exposes personally identifiable information (PII) to services that do not require it, violating the principle of least privilege and data minimization.⁶
- **Lifecycle Desynchronization:** Access tokens are configured with very short lifetimes (e.g., 5 to 15 minutes) to reduce the window of opportunity if a token is leaked.³ Conversely, because user session durations in client interfaces can be longer, ID Tokens are often issued with extended lifetimes.³ Using an ID Token as an API credential extends the exposure window of the system's entry points.³

In the hybrid flow of OIDC, specific security claims are embedded within the ID Token to tie these lifecycles together cryptographically.²⁸ The at_hash (Access Token Hash) claim is a base64url-encoded representation of the left-most half of the SHA-256 hash of the Access Token.²⁸ Similarly, the c_hash (Code Hash) represents the left-most half of the SHA-256 hash of the authorization code.²⁸ These claims allow the client to verify that the Access Token or Authorization Code returned from the front-channel was not tampered with or substituted.²⁸

The following table details the technical differences between these two tokens:

Feature/Dimension	Access Token (OAuth 2.0)	ID Token (OpenID Connect Core 1.0)
Primary Cryptographic Purpose	Delegation of authority to call protected endpoints on behalf of the user. ³	Verification of user identity and the authentication event. ³
Intended Audience (Audience Claim Validation)	Consumed by the Resource Server (API); aud must match the API's identifier. ³	Consumed by the Relying Party (Client); aud must match the client_id. ⁷
Structural Standard (Opaque vs. JWT)	Undefined by standard; can be opaque references, database keys, or structured JWTs. ²⁴	MUST be structured as a cryptographically signed JSON Web Token (JWT). ²⁸

Common Scopes	API-specific granular actions (e.g., orders.read, payments.write, admin:delete). ⁷	Standardized OIDC identity scopes (e.g., openid, profile, email, address, phone). ⁷
Transmission/Consumption Mechanism	Included in the HTTP Authorization header as a bearer token. ⁷	Parsed and processed locally by client code to establish application state and user sessions. ³
Standard Claims Contained	scope, client_id, exp, iss, aud, and custom permission metadata. ⁷	iss, sub, aud, exp, iat, auth_time, nonce, acr, amr, at_hash. ⁷
Verification Authority	Resource Server (statelessly via public keys or dynamically via introspection). ³	Relying Party (Client-side validation of signature, issuer, audience, and session nonce). ³

Deep Cryptographic Anatomy of JSON Web Tokens (JWT - RFC 7519)

A JSON Web Token (JWT) is an open, standardized container format (RFC 7519) used to transmit claims between distributed systems as a URL-safe, compact string.¹⁸ A compact JWT consists of three distinct segments, separated by periods (.): Header, Payload, and Signature.¹⁸

JOSE Header Architecture

The JSON Object Signing and Encryption (JOSE) Header contains the metadata that defines the cryptographic operations applied to the token's payload.¹² It is represented as a JSON object with several standard properties¹⁸:

- **alg (Algorithm)**: Declares the cryptographic algorithm used to sign the token.¹⁸ Standard options include asymmetric algorithms such as RS256 (RSASSA-PKCS1-v1_5 using SHA-256)¹⁸, ES256 (ECDSA using the P-256 curve and SHA-256)²⁰, or symmetric algorithms like HS256 (HMAC using SHA-256).¹⁸
- **typ (Type)**: Declares the media type of the token, typically set to JWT.²⁷
- **kid (Key ID)**: A unique identifier indicating which key within the Authorization Server's public key set (JWKS) was used to sign the token.²⁰

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "key_2026_id_01"
}
```

Header Security Warning: The JOSE Header is transformed into its wire format using Base64Url encoding, which simply strips padding characters (=) and replaces URL-unsafe characters (+ to -, / to _).¹⁸ It is not encrypted.²¹ This encoding provides no cryptographic confidentiality.¹⁸ Any intermediate network node or client can decode this segment and read the algorithms or key IDs.²¹ Resource Servers must not trust any parameter in this header until the signature is verified.²⁷

Payload Claims Set

The Payload contains the actual claims—assertions made about the subject and additional metadata.¹⁸ These claims are divided into three namespaces²⁸:

Registered Claims

These standard, pre-defined claims should be validated by the recipient to ensure token integrity and prevent lifecycle attacks²⁰:

- **iss (Issuer):** A case-sensitive string containing a URL that identifies the identity provider that issued the JWT.⁷
- **sub (Subject):** A unique, stable identifier representing the resource owner.⁷ This value must be unique within the issuer's namespace and must not be reassigned.²⁸
- **aud (Audience):** A string or array of strings identifying the intended recipients of the JWT.⁶
- **exp (Expiration Time):** A Unix timestamp defining when the token becomes invalid.⁷
- **nbf (Not Before):** A Unix timestamp representing the earliest time at which the token can be accepted for processing.²⁰
- **iat (Issued At):** A Unix timestamp indicating when the token was created.⁷

Public Claims

Claims that are registered in the IANA JSON Web Token Claims registry or defined using collision-resistant names (such as URIs).³¹ Examples include standard OIDC identity fields, such as email, given_name, and locale.³¹

Private/Custom Claims

Organization-specific claims used to communicate custom access context, such as user roles, tenant IDs, or granular enterprise permissions (e.g., roles, permissions, tenant_id).⁶

```
{
  "iss": "https://auth.enterprise.com/oauth2",
  "sub": "user_1029384756",
  "aud": "https://api.enterprise.com/v2",
  "exp": 1780012000,
  "nbf": 1780011000,
  "iat": 1780011000,
  "roles": ["financial_analyst", "auditor"],
  "tenant_id": "tenant_prod_west_01"
}
```

Payload Security Warning: The Payload is merely Base64Url-encoded and lacks confidentiality.¹⁸ Storing sensitive data, such as unhashed passwords, personally identifiable information (PII), or database keys inside a signed JWT represents a high-risk security vulnerability.³⁷ If the token is logged, intercepted, or leaked to browser storage, this sensitive data is immediately compromised.⁵ For scenarios requiring confidentiality, the token must be encrypted using JSON Web Encryption (JWE).²⁸

Cryptographic Signature and Verification Mechanics

The signature verifies that the token's sender is authentic and that the payload has not been modified in transit.¹⁸

Let H_{b64} be the Base64Url-encoded string representation of the JOSE Header and P_{b64} be the Base64Url-encoded string representation of the Payload Claims Set. The server prepares the cryptographic signing input, S_{input} , by concatenating these two components with a period separator²¹:

$$S_{input} = H_{b64} \parallel "." \parallel P_{b64}$$

The cryptographic operations differ depending on the selected trust model¹⁸:

Symmetric Cryptography (HS256)

Symmetric signature generation uses HMAC-SHA-256.²² A single secret key, K_{shared} , is shared between the issuer and all verifiers.¹⁸ The signature is computed as:

$$\text{Sig}_{\text{HS256}} = \text{Base64Url}(\text{HMAC-SHA-256}(K_{shared}, S_{input}))$$

Under this symmetric trust model, any Resource Server capable of *verifying* a token is also capable of *signing* and generating valid tokens for the entire ecosystem.²² If an attacker compromises a single downstream microservice that stores K_{shared} , they can forge administrative claims and execute privilege-escalation attacks across the entire trust domain.²²

Asymmetric Cryptography (RS256)

Asymmetric signature generation uses RSASSA-PKCS1-v1_5 with SHA-256.¹⁸ This model uses a mathematically linked private-public key pair.¹⁸ The Authorization Server holds the private key, $K_{private}$, and uses it to sign the SHA-256 hash of the input²¹:

$$\text{Sig}_{\text{RS256}} = \text{Base64Url}(\text{Sign}_{K_{private}}(\text{SHA-256}(S_{input})))$$

The Resource Server validates the signature using the corresponding public key, K_{public} .¹⁸ Because the private key is held securely by the Authorization Server, compromising a downstream Resource Server does not allow an attacker to forge tokens.²²

The JSON Web Key Set (JWKS) and Verification Flow

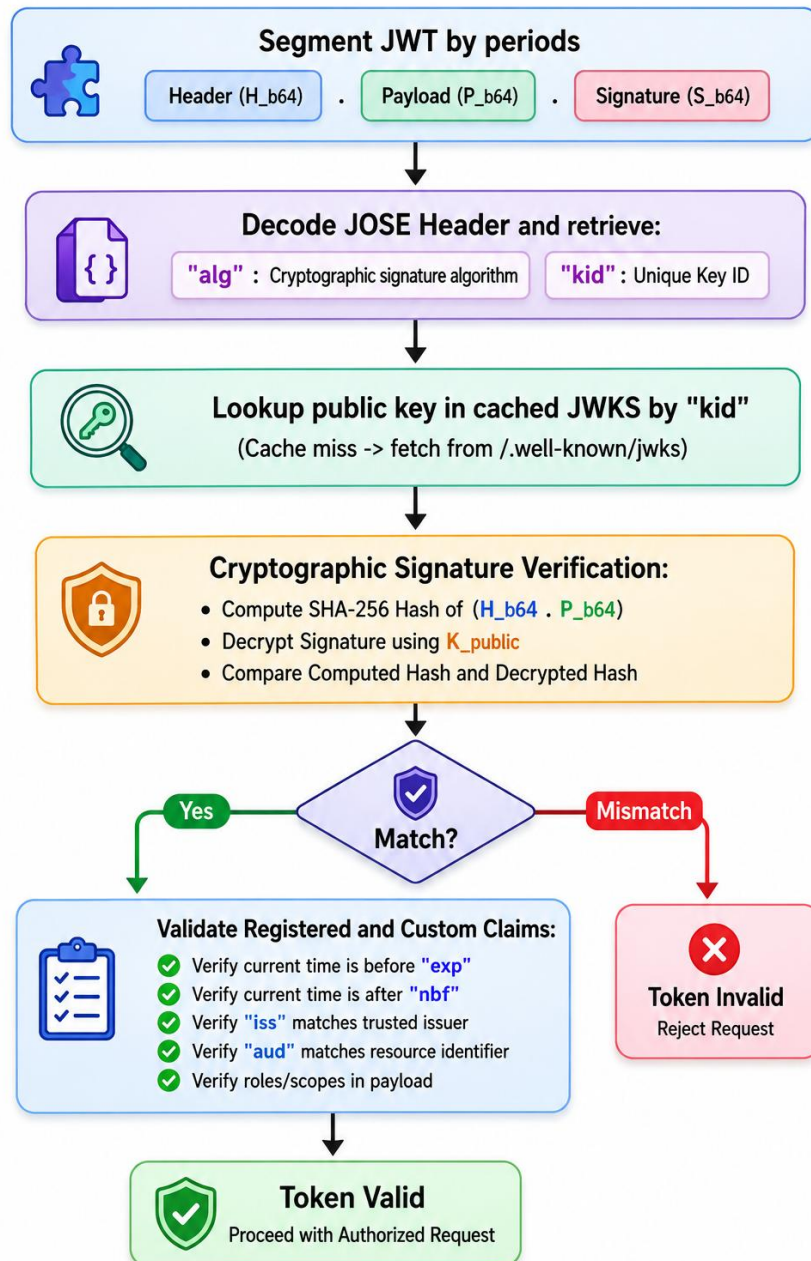
To perform independent, stateless verification of asymmetric signatures, Resource Servers use the JSON Web Key Set (JWKS) standard (RFC 7517).²⁰ A JWKS document contains a keys array of JSON Web Keys (JWK) representing the active public keys of the issuer ²⁰:

```
{
  "keys": [
    {
      "e": "AQAB"
    }
  ]
}
```

The fields within a JWK entry are defined as follows ²⁰:

- **kty (Key Type)**: Identifies the cryptographic algorithm family (e.g., RSA or EC).²⁰
- **use**: Identifies the intended use of the key; sig indicates signature verification, and enc indicates encryption.²⁰
- **n and e**: The modulus and exponent parameters used to reconstruct the RSA public key.²⁰ For Elliptic Curve (EC) keys, these parameters are replaced by the curve identifier crv (e.g., P-256) and the coordinate markers x and y.²⁰

When a Resource Server intercepts an incoming JWT, it executes a standardized five-step signature and validation flow ¹¹:



1. **Token Segmentation:** The server parses the incoming request's Authorization: Bearer <token> header, splitting the JWT at its period boundaries.¹¹
2. **JOSE Analysis:** The server decodes the Base64Url header segment to extract the algorithm (alg) and Key ID (kid).²⁰
3. **Key Retrieval:** The server looks up the corresponding public key in its local cache using the kid.²⁰ If the key is not cached, the server performs a back-channel request to the trusted /.well-known/jwks.json endpoint to update its cache, allowing for key rotation.²⁰
4. **Signature Verification:** The server computes the SHA-256 hash of the combined header and payload (S_{input}).²⁰ It then decrypts the token's signature using K_{public} and compares the two hashes.²⁰ If the computed hash and the decrypted signature do

not match, the server rejects the request.²⁰

5. **Claims Evaluation:** After signature verification, the server parses the payload and verifies that the iss, aud, and exp claims are valid.²⁰ It then evaluates any user roles or scopes embedded in the custom claims to enforce access control.⁷

This verification flow thwarts token tampering and privilege-escalation attacks.²⁷ If an attacker attempts to modify a role in the payload (e.g., from "roles": ["user"] to "roles": ["admin"]) ²⁷, the SHA-256 hash of the modified payload will not match the signature.²¹ Since

the attacker does not possess the private signing key (K_{private}), they cannot generate a matching signature, and the Resource Server will reject the modified token.²¹

Deep Analysis of JWT Vulnerabilities and Mitigations

Implementing JWT-based security without proper architectural controls can expose systems to several notable vulnerabilities:

- **Signature Exclusion (alg: none Vulnerability):** Some early parsing libraries used the token's header to determine how to verify its signature.²⁷ If an attacker changed the alg parameter to none and stripped the signature segment, vulnerable parsers would accept the token as valid without verifying its payload.²⁷ To mitigate this vulnerability, Resource Servers must be configured with a hardcoded list of allowed signature algorithms (e.g., strictly allowing only RS256 or ES256) and must reject tokens with alg: none case-insensitively.²⁷
- **HMAC Key Confusion Attacks:** In this attack, a malicious client exploits a Resource Server that supports both symmetric and asymmetric algorithms.³³ The attacker obtains the Authorization Server's public RSA key (which is publicly available via the JWKS endpoint).¹⁸ They modify the JWT header's alg parameter to HS256 and sign the modified payload using the public RSA key *as the symmetric shared secret*.³³ If the Resource Server's parser is vulnerable, it will process the token using HMAC-SHA-256, verify the signature using the public key as the secret, and accept the forged token.³³ To prevent key confusion, Resource Servers must explicitly bind validation keys to their intended algorithms and must never use asymmetric public keys within symmetric HMAC validation engines.²⁰
- **Key ID (kid) Manipulation:** If a verification engine uses the kid parameter to dynamically retrieve keys from local file systems or databases without proper sanitization, attackers can exploit this path.³³ For example, injecting a directory traversal sequence (e.g., kid: "../../../dev/null") can force the engine to look up a key from an empty file, allowing the attacker to verify the signature using an empty string as the secret key.³³ Additionally, unvalidated kid values can expose systems to SQL or command injection.³³ Resource Servers must sanitize all header parameters and match kid values against a strict list of allowed public key IDs.²⁰
- **Dynamic Header Claims Injection (jku and x5u):** The JSON Web Signature (JWS) standard allows tokens to include header claims that point to signing keys, such as jku (JSON Web Key Set URL) or x5u (X.509 Certificate URL).³³ If a Resource Server dynamically fetches public keys from the URL specified in these headers, an attacker can sign a token with their own private key, set the jku header to point to their own public JWKS endpoint, and bypass signature verification.³³ Resource Servers must disable dynamic key fetching from arbitrary header URLs and must retrieve signing

keys only from pre-configured, trusted domain allowlists.³³

Standard-Compliant Hardening: Sender-Constrained Tokens

Traditional bearer tokens act as "holder" credentials: anyone who presents the token can access the corresponding resources, making them vulnerable to interception, log leakage, or man-in-the-middle attacks.³ To address this risk, modern security standards specify sender-constraining mechanisms that cryptographically bind tokens to the client's key material⁴¹:

- **Demonstrating Proof-of-Possession (DPoP - RFC 9449)**: An application-layer mechanism where the client generates an asymmetric key pair and signs a short-lived DPoP proof JWT for every token request and API call.¹² The Authorization Server binds the issued token to the client's public key by adding a confirmation (cnf) claim containing the key's SHA-256 thumbprint (jkt).⁴¹ The Resource Server then verifies that the thumbprint of the public key in the DPoP proof matches the cnf claim bound to the token, ensuring that intercepted tokens cannot be replayed without the client's private key.⁴¹
- **Mutual TLS Client Certificate-Bound Access Tokens (RFC 8705)**: A transport-layer alternative that binds issued tokens to the client's TLS certificate.⁴² The Authorization Server records the certificate's SHA-256 thumbprint in the cnf.x5t#S256 claim.⁴² Downstream Resource Servers then verify that the client certificate presented during the TLS handshake matches the thumbprint in the token.⁴² While mTLS requires end-to-end TLS termination at the API layer, it provides a robust, zero-trust network control path for backend-to-backend integrations.⁴²

References

1. [D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," IETF, RFC 6749, Oct. 2012. \[Online\]. Available: <https://tools.ietf.org/html/rfc6749>](#)
2. [M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token \(JWT\)," IETF, RFC 7519, May 2015. \[Online\]. Available: <https://tools.ietf.org/html/rfc7519>](#)
3. [N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 2," OpenID Foundation, Dec. 2023. \[Online\]. Available: \[https://openid.net/specs/openid-connect-core-1_0.html\]\(https://openid.net/specs/openid-connect-core-1_0.html\)](#)
4. [T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best Current Practice for OAuth 2.0 Security," IETF, BCP 240, RFC 9700, Jan. 2025. \[Online\]. Available: <https://tools.ietf.org/html/rfc9700>](#)
5. [OWASP Foundation, "OWASP Top 10:2021 – A01:2021 – Broken Access Control," late 2021 / late 2025 official final release, Dec. 2021. \[Online\]. Available: \[https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/\]\(https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/\)](#)
6. [M. Jones, "JSON Web Key \(JWK\)," IETF, RFC 7517, May 2015. \[Online\]. Available: <https://tools.ietf.org/html/rfc7517>](#)
7. [J. Richer, Ed., "OAuth 2.0 Token Introspection," IETF, RFC 7662, Oct. 2015. \[Online\]. Available: <https://tools.ietf.org/html/rfc7662>](#)
8. [D. Fett, K. Campbell, and M. Pozzobon, "Demonstrating Proof of Possession \(DPoP\) at the Application Layer," IETF, RFC 9449, Sep. 2023. \[Online\]. Available: <https://tools.ietf.org/html/rfc9449>](#)
9. [B. Campbell, J. Bradley, and J. Richer, "OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens," IETF, RFC 8705, Feb. 2020. \[Online\]. Available: <https://tools.ietf.org/html/rfc8705>](#)

10. [W. Denniss and J. Bradley, "OAuth 2.0 for Native Apps," IETF, BCP 212, RFC 8252, Oct. 2017.](https://tools.ietf.org/html/rfc8252) [Online]. Available: <https://tools.ietf.org/html/rfc8252>

Works cited

1. RFC 6749 - The OAuth 2.0 Authorization Framework - IETF Datatracker, accessed May 28, 2026, <https://datatracker.ietf.org/doc/html/rfc6749>
2. OAuth 2.0 & OpenID Connect: The Complete Guide to What the Standards Actually Say, accessed May 28, 2026, <https://mrutyunjaypatil.medium.com/oauth-2-0-openid-connect-the-complete-guide-to-what-the-standards-actually-say-e92f040a4251>
3. How Access and ID Tokens Flow in OAuth & OIDC | Duende, accessed May 28, 2026, <https://duendesoftware.com/learn/how-access-tokens-and-id-tokens-flow-through-oauth-openid-connect>
4. OAuth best practices: We read RFC 9700 so you don't have to - WorkOS, accessed May 28, 2026, <https://workos.com/blog/oauth-best-practices>
5. A01 Broken Access Control - OWASP Top 10:2021, accessed May 28, 2026, https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/
6. Identity Chaining Deep Dive: Connecting Identity Across Trust Domains with OAuth, accessed May 28, 2026, <https://dev.to/kanywst/identity-chaining-deep-dive-connecting-identity-across-trust-domains-with-oauth-2onb>
7. ID Token vs. Access Token: Understand the Difference - LoginRadius, accessed May 28, 2026, <https://www.loginradius.com/blog/identity/id-token-vs-access-token>
8. OAuth 2.0 Token Introspection (RFC 7662) Explained for APIs and AI Agents - Scalekit, accessed May 28, 2026, <https://www.scalekit.com/blog/oauth-2-0-token-introspection-rfc-7662>
9. RFC 7662 - OAuth 2.0 Token Introspection - IETF Datatracker, accessed May 28, 2026, <https://datatracker.ietf.org/doc/html/rfc7662>
10. RFC 8252: OAuth 2.0 for Native Apps, accessed May 28, 2026, <https://www.rfc-editor.org/rfc/rfc8252.html>
11. RFC6749: The OAuth 2.0 Authorization Framework - Authlib, accessed May 28, 2026, <https://docs.authlib.org/en/v1.5.1/specs/rfc6749.html>
12. RFC 9449 - OAuth 2.0 Demonstrating Proof of Possession (DPoP) - IETF Datatracker, accessed May 28, 2026, <https://datatracker.ietf.org/doc/html/rfc9449>
13. Auth Explained (Part 1): ID vs Access vs Refresh tokens — what they ACTUALLY do (and why localStorage is a trap) - DEV Community, accessed May 28, 2026, <https://dev.to/sylwia-lask/auth-explained-part-1-id-vs-access-vs-refresh-tokens-what-they-actually-do-and-why-211>
14. Updates to OAuth 2.0 Security Best Current Practice - IETF Datatracker, accessed May 28, 2026, <https://datatracker.ietf.org/doc/draft-ietf-oauth-security-topics-update/>
15. Updates to OAuth 2.0 Security Best Current Practice - IETF, accessed May 28, 2026, <https://www.ietf.org/archive/id/draft-ietf-oauth-security-topics-update-00.html>
16. RFC 9700: Best Current Practice for OAuth 2.0 Security, accessed May 28, 2026, <https://www.rfc-editor.org/rfc/rfc9700.html>
17. RFC 9700 - Best Current Practice for OAuth 2.0 Security - IETF Datatracker, accessed May 28, 2026, <https://datatracker.ietf.org/doc/rfc9700/>

18. Difference Between RS256 and HS256 JWT Signing Algorithms - Auth0 Support Center, accessed May 28, 2026, <https://support.auth0.com/center/s/article/What-s-the-difference-between-RS256-and-HS256-JWT-signing-algorithms>
19. Why Bearer Tokens Are No Longer Enough: Secure Your Identity Layer with DPOP (RFC 9449) : r/KeyCloak - Reddit, accessed May 28, 2026, https://www.reddit.com/r/KeyCloak/comments/1seydq8/why_bearer_tokens_are_no_longer_enough_secure/
20. JWKS URL: What It Is, How to Find Yours, and How JWT Validation Uses It - Security Boulevard, accessed May 28, 2026, <https://securityboulevard.com/2026/05/jwks-url-what-it-is-how-to-find-yours-and-how-jwt-validation-uses-it/>
21. RS256 vs HS256 What's the difference? | Auth0, accessed May 28, 2026, <https://auth0.com/blog/rs256-vs-hs256-whats-the-difference/>
22. RS256 vs HS256 - Understanding the Difference in JWT Signing, accessed May 28, 2026, <https://supertokens.com/blog/rs256-vs-hs256>
23. RS256 vs HS256: A deep dive into JWT signing algorithms - WorkOS, accessed May 28, 2026, <https://workos.com/blog/rs256-vs-hs256-jwt-signing-algorithms>
24. OAuth 2.0 Token Introspection - LDAPWiki, accessed May 28, 2026, <https://ldapwiki.com/wiki/Wiki.jsp?page=OAuth%202.0%20Token%20Introspection>
25. RFC7662: OAuth 2.0 Token Introspection - Authlib 1.6.0 documentation, accessed May 28, 2026, <https://docs.authlib.org/en/v1.6.0/specs/rfc7662.html>
26. OAuth 2.0 Token Introspection for B2C: Consumer API Security | Authority Hub - MojoAuth, accessed May 28, 2026, <https://mojoauth.com/ciam-101/oauth-2-0-token-introspection-b2c-consumer-api-security>
27. Analyzing Broken User Authentication Threats to JSON Web Tokens | Akamai, accessed May 28, 2026, <https://www.akamai.com/blog/security-research/owasp-authentication-threats-for-json-web-token>
28. OpenID Connect Core 1.0 Deep Dive: Understanding the "Authentication" Layer on top of OAuth 2.0 - DEV Community, accessed May 28, 2026, <https://dev.to/kanywst/openid-connect-core-10-deep-dive-understanding-the-authentication-layer-on-top-of-oauth-20-2bei>
29. Identity tokens vs Access tokens: understanding the key differences - WorkOS, accessed May 28, 2026, <https://workos.com/blog/identity-tokens-vs-access-tokens-understanding-the-key-differences>
30. A01 Broken Access Control - OWASP Top 10:2025, accessed May 28, 2026, https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/
31. OpenID Connect 1.0 - Orange Developer, accessed May 28, 2026, <https://developer.orange.com/resources/openid-connect-1-0/>
32. JSON Web Token for Java - OWASP Cheat Sheet Series, accessed May 28, 2026, https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
33. Testing JSON Web Tokens - WSTG - Latest | OWASP Foundation, accessed May 28, 2026, https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/10-Testing_JSON_Web_Tokens
34. Publish your JSON Web Key Set (JWKS) - GOV.UK Wallet Technical

- Documentation, accessed May 28, 2026, <https://docs.wallet.service.gov.uk/issue-credentials/jwks/>
35. Keys to JWT Assessments - From a Cheat Sheet to a Deep Dive - TrustedSec, accessed May 28, 2026, <https://trustedsec.com/blog/keys-to-jwt-assessments-from-a-cheat-sheet-to-a-deep-dive>
 36. OpenID Connect for Agents (OIDC-A) 1.0: A Standard Extension for LLM-Based Agent Identity and Authorization - arXiv, accessed May 28, 2026, <https://arxiv.org/pdf/2509.25974>
 37. Update: Authentication Cheat Sheet · Issue #1973 · OWASP/CheatSheetSeries - GitHub, accessed May 28, 2026, <https://github.com/OWASP/CheatSheetSeries/issues/1973>
 38. Comparison of RS256 and HS256 Algorithms for Token Signing in Cryptography - Sreyanth, accessed May 28, 2026, <https://bvsreyanth.medium.com/comparison-of-rs256-and-hs256-algorithms-for-token-signing-in-cryptography-bd21e9e7a54d>
 39. RFC 7517 - JSON Web Key (JWK) - IETF Datatracker, accessed May 28, 2026, <https://datatracker.ietf.org/doc/html/rfc7517>
 40. JWT SECURITY, accessed May 28, 2026, <https://owasp.org/www-chapter-belgium/assets/2021/2021-02-18/JWT-Security.pdf>
 41. DPOP (RFC 9449) explained: How sender-constrained OAuth tokens make token theft a non-event - WorkOS, accessed May 28, 2026, <https://workos.com/blog/dpop-rfc-9449-explained>
 42. Sender-constrained tokens: Why mTLS and DPOP exist, and what killed Token Binding, accessed May 28, 2026, <https://workos.com/blog/mtls-dpop-token-binding-sender-constrained-oauth>
 43. Demonstrating Proof of Possession (DPoP) - Thales Docs, accessed May 28, 2026, <https://www.thalesdocs.com/oip/idaas-core/idaas-core-access/oidc/dpop/index.html>